Simulating the Classical Distributions

A working tour from uniform bits to the t-test,
with the `probsim` C99 library

The `probsim` project

July 2026

Abstract

This paper accompanies `probsim`, a small C99 library that simulates the twelve classical probability distributions and carries the results all the way to working t-tests. For each distribution we explain what it models, where it earns its keep in statistical inference and applied modeling, and how the library simulates it — always with the simplest algorithm that is still practical. The distributions are presented as one connected family (Figure 1): every sampler is built from uniform bits, and the family funnels naturally into Student's t, whose tail areas turn simulated or observed data into p-values. Every section names the source file and function that implements it, and the sources cite these sections back, so paper and code can be read side by side. This paper also has an interactive web edition (`docs/index.html`, served via GitHub Pages) with every figure animated — each figure caption below links to its interactive counterpart.

1 Introduction

A probability distribution is a reusable answer to the question "what does this kind of randomness look like?" A handful of them — the *classical* distributions — cover a remarkable share of practical work: coin-like events, counts, waiting times, measurement noise, and the sampling behaviour of statistics computed from data. Learning them as isolated formulas is hard; learning them as a *family*, where each new member is a short construction from ones you already know, is much easier. That family view is exactly how the `probsim` library is written, and Figure 1 is its map.

How to read this with the code. The library is four small C files behind one header, `include/probsim.h`. Section §2 covers `src/rng.c`; §3 and §4 cover `src/dist.c`; §5 covers `src/special.c` and `src/stats.c`; §6 walks through the driver `app/simulate.c` and the test suite `tests/test_probsim.c`. Each subsection below ends with a pointer to its implementation, and the source comments point back here. Throughout, the guiding rule is the one this library was written under: *prefer the simplest algorithm that is easy to understand and still practical to use.* Where a cleverer method is the industry standard — the ziggurat for normals, say (Marsaglia and Tsang, 2000b) — we cite it, explain the trade-off, and keep the simple one.

2 Where the randomness comes from

Computers are deterministic, so "random" numbers are produced by a *pseudo-random number generator* (PRNG): a small state machine whose output sequence is indistinguishable, for statistical purposes, from true randomness. `probsim` uses **xoshiro256++** (Blackman and Vigna, 2021): 256 bits of state, a period of $2^{256} - 1$, excellent results on the standard test batteries, and a step function of about ten lines — comfortably inside our simplicity rule.

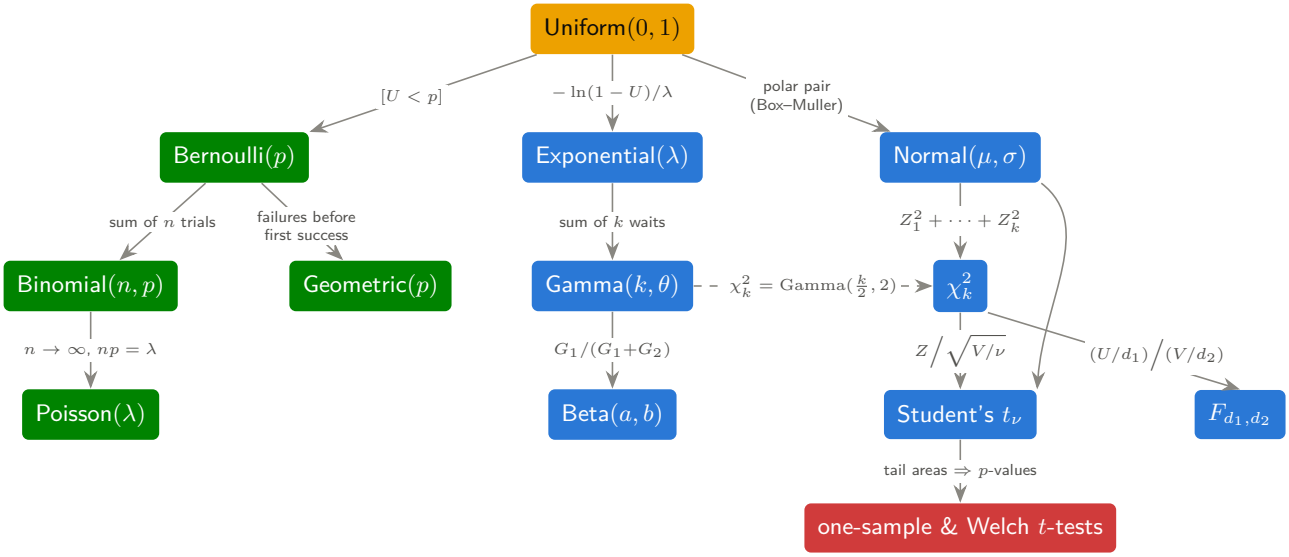

Figure 1: The classical family as `probsim` builds it. Everything starts from uniform bits (§2); discrete distributions grow along the left (§3), continuous ones along the right (§4), and the family funnels into the t-tests of §5. Each arrow is literally a line or two of C in `src/dist.c`. [interactive version on the web]

Reasonable alternatives with the same spirit include PCG (O'Neill, 2014); the deeper theory of testing generators is in Knuth (1997, ch. 3). None of these are cryptographic generators, and `probsim` should never be used where an adversary matters.

Two practical details deserve attention because they bite working statisticians. First, *seeding*: naive seeds like $1, 2, 3$ are nearly identical bit patterns, and a weak seeding routine would start the streams in correlated states. The library therefore runs every seed through the splitmix64 mixer, as the xoshiro authors recommend. Second, *explicit state*: every sampler takes a `ps_rng *` argument and there is no hidden global, so a simulation is reproducible from a single recorded seed and independent streams are just independent structs — which is what makes the test suite of §6 deterministic.

2.1 Uniform is enough

The raw generator emits 64-bit integers; dividing the top 53 bits by 2^{53} gives a double U uniform on $[0, 1)$ at full mantissa precision. Everything else in this paper is manufactured from such U's. The master tool is *inverse-transform sampling*: if F is a CDF and $U \sim \text{Uniform}(0, 1)$, then $X = F^{-1}(U)$ has CDF exactly F, because $\Pr[X \le x] = \Pr[U \le F(x)] = F(x)$. Figure 2 shows the idea; the exponential sampler of §4.2 is its purest one-line application. When F^{-1} is awkward, two other levers — *transformation* (build the variable from simpler ones) and *rejection* (propose, then accept with the right probability) — take over; both appear in §4. The encyclopedia of all three techniques is Devroye (1986).

↪ implemented in `src/rng.c`: `ps_rng_seed()`, `ps_rng_next()`, `ps_runif()`

3 The discrete family

Discrete distributions describe counts. The four classical ones form a single story: a yes/no event (Bernoulli), how many yeses in n tries (binomial), how long until the first yes (geometric), and how many events occur when opportunities are numerous but each is rare (Poisson). The driver `app/simulate.c` exercises each with the worked parameters used below.

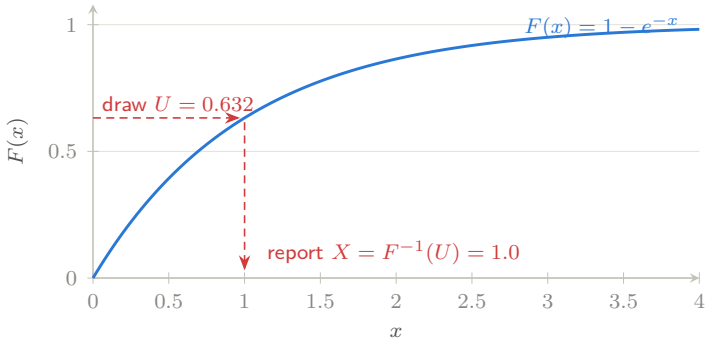

Figure 2: Inverse-transform sampling: a uniform draw on the vertical axis is pushed through F^{-1} to land on the horizontal axis with density exactly F'. Regions where F rises steeply receive more of the uniform mass — that is the whole trick. [interactive version on the web]

3.1 Bernoulli

The Bernoulli(p) variable is a single yes/no experiment: $\Pr[X{=}1] = p$, $\Pr[X{=}0] = 1 - p$, with $\mathbb{E}X = p$ and $\operatorname{Var} X = p(1-p)$. It is the atom of applied statistics: a conversion in an A/B test, a defective part, a patient responding to treatment, a loan defaulting. Almost all of *binary-outcome* inference — estimating a proportion, comparing two proportions, logistic regression — is Bernoulli modeling. Simulation is the definition itself: draw U and report whether $U < p$.

↪ implemented in `src/dist.c`: `ps_bernoulli()`

3.2 Binomial

The Binomial(n, p) variable counts successes in n independent Bernoulli(p) trials:

$$\Pr[X = k] = \binom{n}{k} p^k (1-p)^{n-k}, \qquad \mathbb{E}X = np, \quad \operatorname{Var} X = np(1-p).$$

It models any fixed-quota count: heads in n tosses, positive responses among n patients, defects in a batch of n (the basis of industrial *acceptance sampling*), conversions among n visitors. In inference it is the exact sampling distribution behind every test of a proportion — the sign test, the exact binomial test, and, through the normal approximation, the familiar z-test for proportions. `probsim` simulates it as what it is — n Bernoulli draws summed — an $O(n)$ loop that cannot be wrong by inspection; sub-linear samplers exist (Devroye, 1986, ch. X.4) but buy speed with complexity we do not need. Figure 3 (left) shows the worked example Binomial(20, 0.25).

↪ implemented in `src/dist.c`: `ps_binomial()`, `ps_binomial_pmf()`

3.3 Geometric

The Geometric(p) variable counts *failures before the first success* (support $0, 1, 2, \ldots$, the convention of R's `rgeom`): $\Pr[X = k] = p(1-p)^k$, $\mathbb{E}X = (1-p)/p$, $\operatorname{Var} X = (1-p)/p^2$. It is the discrete waiting time: retries before a packet gets through, cold calls before a sale, coin flips before the first head. Its *memoryless* property (past failures tell you nothing about the future) makes it the discrete twin of the exponential — and indeed the sampler exploits exactly that: invert the geometric CDF with one uniform, $X = \lfloor \ln(1-U)/\ln(1-p) \rfloor$, which is the exponential inversion of §4.2 rounded down to a count.

↪ implemented in `src/dist.c`: `ps_geometric()`, `ps_geometric_pmf()`

3.4 Poisson

The Poisson(λ) variable is the law of rare events: the limit of Binomial($n, \lambda/n$) as $n \to \infty$,

$$\Pr[X = k] = e^{-\lambda}\frac{\lambda^k}{k!}, \qquad \mathbb{E}X = \operatorname{Var} X = \lambda.$$

It models counts per unit of exposure when events are individually unlikely but opportunities are many: arrivals at a queue per minute, radioactive decays per second, typos per page, insurance claims per policy-year, sequencing reads per gene. In inference it anchors count regression (Poisson GLMs, with the $\mathbb{E}X = \operatorname{Var} X$ property serving as the canonical check for *overdispersion*) and classical goodness-of-fit tests. The library simulates it with Knuth's product-of-uniforms method (Knuth, 1997, §3.4.1) — count uniforms until their running product drops below $e^{-\lambda}$ — applied in chunks of rate at most 30 so the product never underflows; the chunking is valid because Poisson rates add. Figure 3 (right) shows Poisson(4).

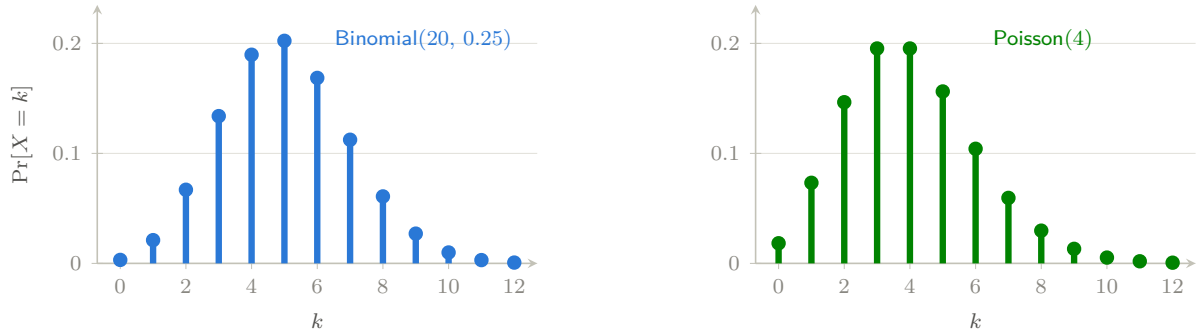

Figure 3: The two classical count laws, at the driver's worked parameters. Both have mean ≈ 4–5, but the binomial is confined to $\{0, \ldots, 20\}$ while the Poisson has an unbounded right tail — the visible signature of "many opportunities, each rare". [interactive version on the web]

↪ implemented in `src/dist.c`: `ps_poisson()`, `ps_poisson_pmf()`

4 The continuous family

Continuous distributions describe measurements. We meet them in the order the library constructs them (the right-hand side of Figure 1), because each is a short recipe from the previous ones.

4.1 Uniform

Uniform(a, b) spreads its mass evenly: density $1/(b-a)$ on $[a, b)$, mean $(a+b)/2$, variance $(b-a)^2/12$. Beyond being the raw material of §2.1, it is a model in its own right (rounding error, phases, arrival offsets) and a cornerstone of inference through one beautiful fact: *under a true null hypothesis, a p-value is Uniform*$(0, 1)$. That fact is testable — the test suite verifies it empirically in §6.3 — and it underlies p-value histograms, false-discovery-rate methods, and simulation-based calibration. Simulation is one affine map of U.

↪ implemented in `src/dist.c`: `ps_uniform()`

4.2 Exponential

The Exponential(λ) variable (rate λ; density $\lambda e^{-\lambda x}$, mean $1/\lambda$, variance $1/\lambda^2$) is the continuous waiting time to the next event of a constant-rate process — the continuous twin of the geometric,

and like it *memoryless*: a component that has survived t hours is statistically as good as new. That property makes it the baseline model of reliability engineering, queueing theory (service and inter-arrival times), and survival analysis, where its constant *hazard rate* is the reference against which ageing (or infant-mortality) effects are measured. The sampler is the one-liner promised in Figure 2: $X = -\ln(1-U)/\lambda$.

↪ implemented in `src/dist.c`: `ps_exponential()`, `ps_exponential_cdf()`

4.3 Normal

The Normal(μ, σ) density $\frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2}$ is the center of gravity of statistics, for one overwhelming reason: the *central limit theorem*. Sums and averages of many small independent effects are approximately normal almost regardless of the ingredients' own distributions — which is why measurement errors, biological traits, and sample means all look Gaussian, and why the normal is the default model for noise in regression and the reference distribution for estimators. Essentially all of §5 exists to make small-sample corrections to normal-based reasoning.

F^{-1} has no closed form, so inversion is unavailable; instead the library uses the polar variant of the Box–Muller transform (Box and Muller, 1958), our one *rejection* sampler (Figure 4): draw (v_1, v_2) uniformly in the square $[-1, 1]^2$, accept when $s = v_1^2 + v_2^2 < 1$ (probability $\pi/4 \approx 79\%$), and return $v_1\sqrt{-2\ln s/s}$, which is exactly standard normal.[1]

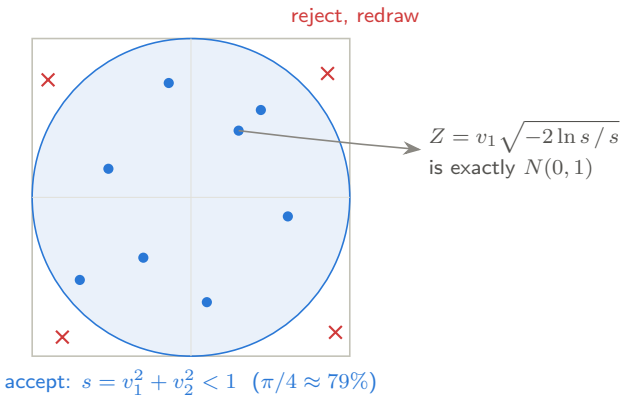

Figure 4: The polar Box–Muller sampler: uniform points in the square are kept only if they land in the disc; the kept point's radius and direction are then reshaped into a standard normal. Rejection turns an easy distribution (uniform on a square) into a hard one (uniform on a disc) at a known, modest cost. [interactive version on the web]

↪ implemented in `src/dist.c`: `ps_normal()`, `ps_normal_pdf()`, `ps_normal_cdf()`

4.4 Gamma

The Gamma(k, θ) family (shape k, scale θ; density $\propto x^{k-1}e^{-x/\theta}$, mean $k\theta$, variance $k\theta^2$) is the flexible law of *accumulated* waiting: for integer k it is exactly the time until the k-th event of a Poisson process — a sum of k exponentials — and general k interpolates smoothly (Figure 5). It models rainfall totals, insurance losses, service times, and neuronal inter-spike intervals; in Bayesian inference it is the conjugate prior for Poisson and exponential rates, so posterior

[1] The method actually yields *two* independent normals, $v_1\sqrt{-2\ln s/s}$ and $v_2\sqrt{-2\ln s/s}$; `ps_normal()` discards the second so the sampler carries no state between calls. Production libraries chasing speed instead use the ziggurat method (Marsaglia and Tsang, 2000b), a heavily engineered rejection scheme — a good example of the complexity our simplicity rule declines.

simulation of rate parameters is gamma sampling. It also generates the next two distributions outright.

Summing exponentials only works for integer k, so the library uses the one algorithm where we accept a little cleverness for a lot of practical value: the Marsaglia and Tsang (2000a) method, the standard choice of modern statistical software, and still only a dozen lines. Its idea is a polished version of Figure 4's: push a normal draw Z through the smooth cube $X = d(1 + Z/\sqrt{9d})^3$ with $d = k - \frac{1}{3}$, which lands *almost* exactly on the gamma density, then repair the small discrepancy with a rejection step that accepts well over 95% of proposals. Shapes $k < 1$ (where the density blows up at zero) are handled by a classical boost: draw at shape $k + 1$ and multiply by $U^{1/k}$.

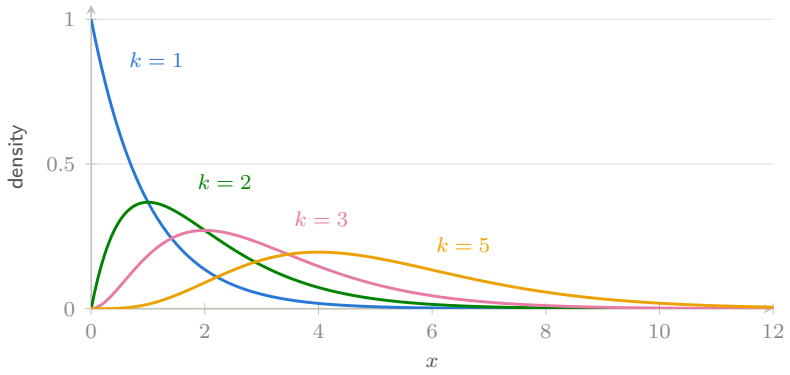

Figure 5: Gamma densities with scale $\theta = 1$. Shape $k = 1$ is the exponential; as k grows the density moves right, symmetrizes, and (by the CLT, since it is a sum of k waits) approaches a normal — the family tree of Figure 1 in a single picture. [interactive version on the web]

↪ implemented in `src/dist.c`: `ps_gamma()`, `ps_gamma_cdf()`

4.5 Beta

The Beta(a, b) family lives on $(0, 1)$ with density $\propto x^{a-1}(1-x)^{b-1}$, mean $a/(a+b)$, and variance $ab/((a+b)^2(a+b+1))$; its shapes (Figure 6) run from uniform through unimodal humps to J- and U-curves. It is *the* distribution for unknown proportions: in Bayesian inference it is the conjugate prior for the Bernoulli/binomial p — start from Beta(a, b), observe s successes and f failures, and the posterior is Beta$(a+s, b+f)$, which is why Thompson-sampling bandits and Bayesian A/B testing are, computationally, beta simulation. It also models proportions directly (beta regression) and is the order-statistics law: the k-th smallest of n uniforms is Beta$(k, n-k+1)$. The sampler uses the classical gamma representation: $X = G_1/(G_1 + G_2)$ with independent $G_1 \sim \text{Gamma}(a, 1)$, $G_2 \sim \text{Gamma}(b, 1)$.

↪ implemented in `src/dist.c`: `ps_beta()`, `ps_beta_cdf()`

4.6 Chi-squared

The χ^2_k variable is a sum of k squared standard normals; it is also exactly Gamma$(k/2, 2)$, which is how the library both simulates it (one call to `ps_gamma`) and evaluates its CDF. Mean k, variance $2k$. Its starring role is as the *distribution of accumulated squared noise*: in a normal sample of size n, the rescaled sample variance $(n-1)S^2/\sigma^2$ is χ^2_{n-1} — the fact that makes confidence intervals for variances possible and, in the next section, gives the t statistic its denominator. It reappears wherever squared deviations are summed: Pearson's goodness-of-fit and independence tests, and the asymptotic null distribution of likelihood-ratio statistics (Wilks' theorem) — which makes χ^2 tail areas the common currency of model comparison.

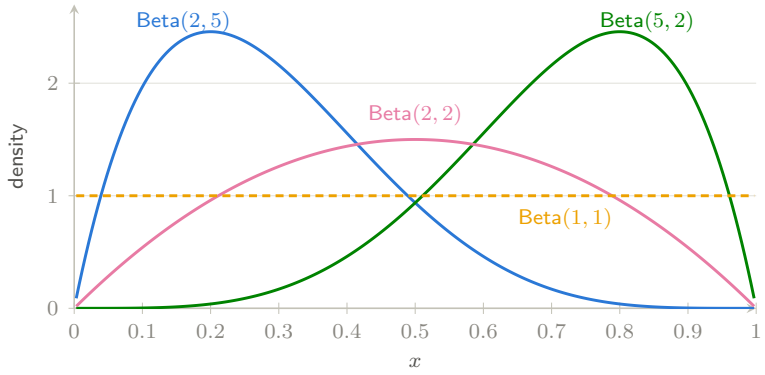

Figure 6: Beta densities: the driver's worked example Beta(2, 5) leans toward small proportions, its mirror Beta(5, 2) toward large ones, Beta(2, 2) is a symmetric hump, and Beta(1, 1) is the uniform — the natural "know nothing" prior for a probability. [interactive version on the web]

↪ implemented in `src/dist.c`: `ps_chi_squared()`, `ps_chi_squared_cdf()`

4.7 Student's t

Student's t_ν is defined by the construction at the heart of this library:

$$T = \frac{Z}{\sqrt{V/\nu}}, \qquad Z \sim N(0,1), \quad V \sim \chi^2_\nu \text{ independent},$$

and the sampler is that definition verbatim. It looks like a normal but with *heavier tails* (Figure 7): dividing by an estimated, rather than known, scale injects extra uncertainty, and the smaller ν is, the fatter the tails. As $\nu \to \infty$ it converges to the normal; at $\nu = 1$ it is the wild Cauchy with no mean at all. Historically it was discovered by Student (1908) — W. S. Gosset, publishing pseudonymously from the Guinness brewery — precisely because brewery samples were too small for large-sample normal theory. Beyond the t-tests of §5 (and the confidence intervals $\bar{x} \pm t^*_\nu \, s/\sqrt{n}$ dual to them), its heavy tails make it a workhorse *model* in its own right: robust regression replaces normal errors with t errors so outliers stop dragging the fit, and finance uses t innovations because asset returns produce extreme moves far more often than a Gaussian allows.

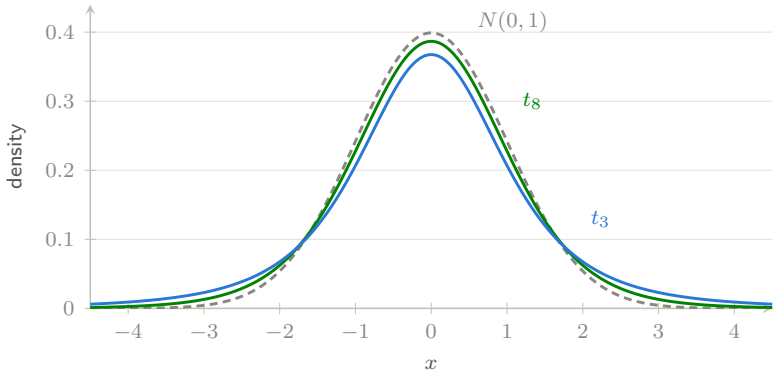

Figure 7: Student's t against the standard normal. The lower the degrees of freedom, the more probability escapes into the tails — so small samples need larger observed effects before a result counts as surprising. This gap between t_ν and $N(0,1)$ *is* the small-sample correction of §5. [interactive version on the web]

↪ implemented in `src/dist.c`: `ps_student_t()`, `ps_student_t_pdf()`, `ps_student_t_cdf()`

4.8 Fisher's F

The F_{d_1,d_2} variable is a ratio of independent normalized chi-squareds, $F = (U/d_1)/(V/d_2)$ — the sampler, once again, is the definition. Since sample variances are scaled chi-squareds (§4.6), F is the natural null distribution for *comparing variances*: its tail areas power the classical ANOVA F-test (is the variance *between* group means larger than the variance *within* groups?), the overall significance test of a regression, and nested-model comparisons. Two connections knit the family shut: $t_\nu^2 = F_{1,\nu}$ (squaring a t-test gives an F-test), and ANOVA with two groups is exactly the pooled t-test.

↪ implemented in `src/dist.c`: `ps_f()`, `ps_f_cdf()`

5 From distributions to inference: the t-test

Everything so far generated data; this section reasons *backwards* from data. The t-test is the classical family's flagship application, and its logic recurs across all of statistics: reduce the data to a statistic whose distribution under a null hypothesis is known exactly (here, thanks to §4.3–§4.7), then ask how extreme the observed value is under that distribution.

5.1 Two special functions carry all the tables

Before software, the answer to "how extreme?" lived in printed tables at the back of textbooks. All of those tables compress into two special functions. The *regularized incomplete beta function*

$$I_x(a,b) = \frac{1}{B(a,b)} \int_0^x u^{a-1}(1-u)^{b-1}\,du, \tag{5.1}$$

is the beta CDF, and the t and F CDFs reduce to it algebraically; for the t,

$$\Pr[T_\nu \le t] \;=\; 1 - \tfrac{1}{2}\, I_{\frac{\nu}{\nu+t^2}}\!\left(\tfrac{\nu}{2}, \tfrac{1}{2}\right) \qquad (t \ge 0), \tag{5.2}$$

with the $t < 0$ case supplied by symmetry, and $\Pr[F_{d_1,d_2} \le x] = I_{d_1 x/(d_1 x + d_2)}(d_1/2,\, d_2/2)$. Its sibling, the *regularized lower incomplete gamma function* $P(a,x) = \frac{1}{\Gamma(a)} \int_0^x u^{a-1} e^{-u}\,du$, is the gamma CDF and hence the chi-squared and Poisson tail machinery. `probsim` evaluates both with the classical series/continued-fraction pairs (Press et al., 1992, §6.2, §6.4), accurate to about 10^{-14} — each roughly forty lines of C that replace every table in the book.

↪ implemented in `src/special.c`: `ps_incbeta()`, `ps_incgamma_lower()`

5.2 The one-sample t-test

Model n observations as $X_i \sim N(\mu, \sigma^2)$ with both parameters unknown, and test $H_0\colon \mu = \mu_0$. Three facts from §4 assemble the test. Under H_0: (i) $\bar{X} \sim N(\mu_0, \sigma^2/n)$; (ii) $(n-1)S^2/\sigma^2 \sim \chi^2_{n-1}$, where S^2 is the sample variance with the $n-1$ divisor — exactly the divisor that makes this distributional statement true, which is why `ps_summarize` uses it; (iii) $\bar{X}$ and S^2 are independent. Standardizing $\bar{X}$ and substituting S for the unknown σ produces precisely the $Z/\sqrt{V/\nu}$ construction of §4.7:

$$T = \frac{\bar{X} - \mu_0}{S/\sqrt{n}} \;\sim\; t_{n-1} \quad \text{under } H_0. \tag{5.3}$$

The unknown σ cancels — that is the miracle Gosset found. The two-sided p-value is then a pure CDF evaluation, $p = 2\,F_{t_{n-1}}(-|t_{\text{obs}}|)$, via equation (5.2):

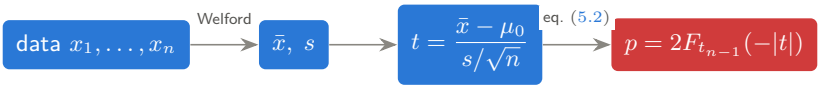

Figure 8 shows the geometry on the library's own test fixture (§6.3): ten measurements with mean 5.47, tested against $\mu_0 = 5$, give $t_{\text{obs}} = 3.12$ on 9 degrees of freedom and $p = 0.012$ — the observed statistic sits well beyond the 5% critical value, so either H_0 is false or a 1-in-80 coincidence occurred. The summaries themselves are computed by Welford's one-pass recurrence (Welford, 1962), which avoids the catastrophic cancellation of the naive sum-of-squares formula.

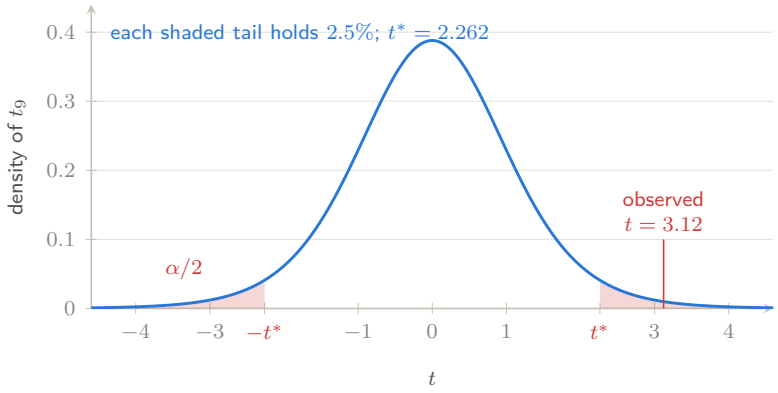

Figure 8: The one-sample t-test on the repository's own test fixture ($n = 10$, df $= 9$). Under H_0 the statistic follows the blue t_9 density; the shaded tails beyond $\pm t^*$ are the 5% rejection region, and the observed $t = 3.12$ lands past them, giving the two-sided p-value 0.012 — the total tail area beyond ± 3.12. [interactive version on the web]

↪ implemented in `src/stats.c`: `ps_summarize()`, `ps_t_test_one_sample()`

5.3 Welch's two-sample t-test

The commonest real question is comparative: do two groups — treatment and control, variant A and variant B — share a mean? With samples $x_1, \ldots, x_{n_x}$ and $y_1, \ldots, y_{n_y}$, Welch (1947) tests $H_0 \colon \mu_X = \mu_Y$ with

$$t = \frac{\bar{x} - \bar{y}}{\sqrt{s_x^2/n_x + s_y^2/n_y}}, \qquad \nu \approx \frac{\left(s_x^2/n_x + s_y^2/n_y\right)^2}{\dfrac{(s_x^2/n_x)^2}{n_x - 1} + \dfrac{(s_y^2/n_y)^2}{n_y - 1}}, \tag{5.4}$$

the degrees of freedom being the Welch–Satterthwaite approximation (Satterthwaite, 1946) — generally fractional, which is why `ps_t_test` carries `df` as a double. Unlike the older pooled test, Welch's does not assume the two variances are equal, and it loses almost nothing when they happen to be; that robustness has made it the default two-sample test in modern practice (and in R's `t.test`). It is the standard analysis for randomized experiments and A/B tests; the same statistic inverted gives the confidence interval for the difference in means that any experiment report should carry.

↪ implemented in `src/stats.c`: `ps_t_test_welch()`

6 The driver and the tests

The repository ships two executables that turn the theory above into running evidence: a driver that *shows* the library working, and a test suite that *proves* it.

6.1 Simulation meets theory

The driver (`make run`) simulates 200,000 draws from each distribution at the worked parameters of §3–§4 and prints sample mean and variance beside the theoretical $\mathbb{E}X$ and $\operatorname{Var} X$ — the law of large numbers as a table (excerpt from seed 20260718):

distribution	mean	$\mathbb{E}X$	variance	$\operatorname{Var} X$
Binomial$(20, 0.25)$	5.008	5	3.733	3.75
Poisson(4)	3.990	4	3.990	4
Exponential(0.5)	1.995	2	3.985	4
Normal$(10, 2)$	10.001	10	3.985	4
Gamma$(3, 2)$	6.009	6	12.002	12
Beta$(2, 5)$	0.285	0.286	0.0254	0.0255
Student's t_8	-0.002	0	1.332	$4/3$
$F_{5,12}$	1.201	1.2	1.077	1.08

6.2 The t-tests, live

The driver then runs both tests of §5 on data it has just simulated — a situation where, uniquely, we *know* the truth. A one-sample test of $N(10, 2)$ data against its true mean (H_0 true) yielded $p = 0.44$: no false alarm. A Welch test of $N(10, 2)$ against $N(11.5, 3)$ (H_0 false) yielded $p = 0.014$: difference detected. Re-running with other seeds shows the expected behaviour: under a true null the p-values scatter uniformly (occasionally below 0.05 — type I error is a feature, not a bug), while under the false null they concentrate near zero at this effect size and n.

6.3 The test suite

`make test` runs about a hundred thousand checks in three layers, in increasing order of subtlety. *Fixtures*: CDF, pmf and t-test results are compared to reference values computed independently with SciPy (each fixture's provenance is recorded beside it in the source). *Identities*: mathematical facts that must hold exactly, such as the symmetry $I_x(a, b)+I_{1-x}(b, a) = 1$ of equation (5.1), or $P(\frac{1}{2}, x) = \operatorname{erf}\sqrt{x}$ linking our incomplete gamma to the C library's `erf`. *Statistical*: with a fixed seed, every sampler's 200,000-draw mean must land within five standard errors of theory; and 2,000 simulated one-sample t-tests under a true null must reject at close to the nominal 5% rate — an end-to-end check that samplers, special functions and test logic agree, and an empirical verification of the uniform-p-value fact of §4.1.

↪ implemented in `app/simulate.c` and `tests/test_probsim.c`

7 A modeler's cheat sheet

The table below compresses the tour into one glance: what each distribution models, and the inference task where it most often appears.

distribution	models	flagship inference role
Bernoulli	single yes/no event	proportion estimation, A/B tests
Binomial	successes in n trials	exact tests of proportions
Geometric	failures until success	retry/attrition modeling
Poisson	rare-event counts	count regression, goodness of fit
Uniform	total ignorance on a range	p-values under H_0; simulation
Exponential	memoryless waiting time	survival & reliability baselines
Normal	sums of small effects	CLT; regression noise
Gamma	accumulated waits, scales	conjugate prior for rates
Beta	unknown proportions	conjugate prior for p; bandits
χ^2	summed squared noise	variance CIs, GoF, likelihood ratios
Student t	normalized small samples	t-tests, robust models
F	variance ratios	ANOVA, regression F-tests

For going further, three references span the territory: Devroye (1986) for everything about generating non-uniform random variables, Knuth (1997) for the theory of the uniform generators underneath, and Press et al. (1992) for the numerical special-function craft of §5.1.

References

David Blackman and Sebastiano Vigna. Scrambled linear pseudorandom number generators. *ACM Transactions on Mathematical Software*, 47(4):1–32, 2021. doi: 10.1145/3460772. Preprint arXiv:1805.01407; local copy: `refs/blackman-vigna-xoshiro.pdf`.

George E. P. Box and Mervin E. Muller. A note on the generation of random normal deviates. *The Annals of Mathematical Statistics*, 29(2):610–611, 1958. doi: 10.1214/aoms/1177706645. Local copy: `refs/box-muller-1958.pdf` (open access).

Luc Devroye. *Non-Uniform Random Variate Generation*. Springer, New York, 1986. doi: 10.1007/978-1-4613-8643-8. Freely available chapter-by-chapter from the author at `luc.devroye.org/rnbookindex.html`.

Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd edition, 1997.

George Marsaglia and Wai Wan Tsang. A simple method for generating gamma variables. *ACM Transactions on Mathematical Software*, 26(3):363–372, 2000a. doi: 10.1145/358407.358414.

George Marsaglia and Wai Wan Tsang. The ziggurat method for generating random variables. *Journal of Statistical Software*, 5(8):1–7, 2000b. doi: 10.18637/jss.v005.i08. Local copy: `refs/marsaglia-tsang-ziggurat-2000.pdf` (open access).

Melissa E. O'Neill. PCG: A family of simple fast space-efficient statistically good algorithms for random number generation. Technical Report HMC-CS-2014-0905, Harvey Mudd College, 2014. Local copy: `refs/oneill-pcg-2014.pdf`.

William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes in C: The Art of Scientific Computing*. Cambridge University Press, 2nd edition, 1992.

Franklin E. Satterthwaite. An approximate distribution of estimates of variance components. *Biometrics Bulletin*, 2(6):110–114, 1946. doi: 10.2307/3002019.

Student. The probable error of a mean. *Biometrika*, 6(1):1–25, 1908. doi: 10.1093/biomet/6.1.1. Local copy: `refs/student-1908.pdf` (public-domain scan).

Bernard L. Welch. The generalization of "Student's" problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, 1947. doi: 10.1093/biomet/34.1-2.28.

B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962. doi: 10.1080/00401706.1962.10490022.